

SUBJECT	OOP-JAVA
NAME	YUG GANDHI
ID	25AIML013

PRACTICAL-1

PROBLEM-1:

(For Advanced Learners – home) Rock-Paper-Scissors-Lizard-Spock (RPSLS.java)

(a) Define an enum Move with ROCK, PAPER, SCISSORS, LIZARD, SPOCK.

(b) Write winner(Move a, Move b) returning 1 if a beats b, -1 if b beats a, 0 for a tie — use a switch expression listing what each move beats (Scissors cut Paper, Paper covers Rock, Rock crushes Lizard, Lizard poisons Spock, Spock smashes Scissors, and the five mirror cases).

(c) In main, play 5 rounds: pick a random computer Move, read the player's Move, call winner(), print the round result and keep score.

(d) After 5 rounds, print the overall winner.

Expected output: *each round prints both moves and the round winner; ends with a “You win 3–2” summary.*

CODE:

```
import java.util.*;

class RPSGame
{
    enum Move { ROCK, PAPER, SCISSORS, LIZARD, SPOCK }

    static int result(Move user, Move computer)
    {
        if (user == computer)
            return 0;

        switch (user)
        {
            case ROCK:
                if (computer == Move.SCISSORS || computer == Move.LIZARD)
                    return 1;
                else
                    return -1;

            case PAPER:
                if (computer == Move.ROCK || computer == Move.SPOCK)
                    return 1;
                else
                    return -1;

            case SCISSORS:
                if (computer == Move.PAPER || computer == Move.LIZARD)
                    return 1;
                else
                    return -1;

            case LIZARD:
                if (computer == Move.ROCK || computer == Move.SPOCK)
                    return 1;
                else
                    return -1;

            case SPOCK:
                if (computer == Move.ROCK || computer == Move.PAPER)
                    return 1;
                else
                    return -1;
        }
    }
}
```

```
        if (computer == Move.PAPER || computer == Move.LIZARD)
            return 1;
        else
            return -1;

    case LIZARD:
        if (computer == Move.PAPER || computer == Move.SPOCK)
            return 1;
        else
            return -1;

    case SPOCK:
        if (computer == Move.ROCK || computer == Move.SCISSORS)
            return 1;
        else
            return -1;
    }

    return 0;
}

public static void main(String args[])
{
    Scanner input = new Scanner(System.in);
    Random random = new Random();

    Move arr[] = Move.values();

    int userScore = 0;
    int computerScore = 0;

    for (int round = 1; round <= 5; round++)
    {
        System.out.println("Round " + round);
        System.out.println("Enter your move:");

        Move user = Move.valueOf(input.next().toUpperCase());
        Move computer = arr[random.nextInt(arr.length)];

        System.out.println("Your Move: " + user);
        System.out.println("Computer Move: " + computer);

        int ans = result(user, computer);

        if (ans == 1)
        {
            System.out.println("You Win!");
            userScore++;
        }
        else if (ans == -1)
        {
            System.out.println("Computer Wins!");
            computerScore++;
        }
        else
        {
            System.out.println("Tie!");
        }
    }
}
```

```
System.out.println("\nFinal Score");
System.out.println("Your Score: " + userScore);
System.out.println("Computer Score: " + computerScore);

if (userScore > computerScore)
    System.out.println("You are the Winner!");
else if (computerScore > userScore)
    System.out.println("Computer is the Winner!");
else
    System.out.println("Match Draw!");
}
```

OUTPUT:

```
Round 1
Enter your move:
ROCK
Your Move: ROCK
Computer Move: SCISSORS
You Win!
Round 2
Enter your move:
SPOCK
Your Move: SPOCK
Computer Move: SCISSORS
You Win!
Round 3
Enter your move:
PAPER
Your Move: PAPER
Computer Move: SPOCK
You Win!
Round 4
Enter your move:
LIZARD
Your Move: LIZARD
Computer Move: SCISSORS
Computer Wins!
Round 5
Enter your move:
SCISSORS
Your Move: SCISSORS
Computer Move: SPOCK
Computer Wins!

Final Score
Your Score: 3
Computer Score: 2
You are the Winner!

=== Code Execution Successful ===
```

PROBLEM-2:**Toll booth (TollBooth.java, main in this file)**

- (a) Define a record Vehicle(String number, String type).
- (b) In main, keep a running toll total and three counters (bike, car, truck).
- (c) Loop until the user types "done" for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type's counter.
- (d) After the loop, print the total toll and the type with the highest count.

Expected output: *two cars + one bike* → "Total toll: 120", "Most frequent: car".

CODE:

```
import java.util.*;

class TollBoothManagement {

    record VehicleDetail(String plateNumber, String vehicleType) {}

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

        int bikeCount = 0;
        int carCount = 0;
        int truckCount = 0;
        int totalToll = 0;

        while (true) {
            System.out.println("Enter vehicle number (or type 'done' to finish):");
            String plate = input.next();

            if (plate.equalsIgnoreCase("done")) {
                break;
            }

            System.out.println("Enter vehicle type:");
            String type = input.next().toLowerCase();

            VehicleDetail currentVehicle = new VehicleDetail(plate, type);

            switch (currentVehicle.vehicleType()) {
                case "bike" -> {
                    totalToll += 20;
                    bikeCount++;
                }
                case "car" -> {
                    totalToll += 50;
                }
            }
        }
    }
}
```

```
        carCount++;
    }
    case "truck" -> {
        totalToll += 150;
        truckCount++;
    }
    default -> System.out.println("Unknown vehicle type. No toll charged.");
}
}

System.out.println("\n--- Summary ---");
System.out.println("Total Toll Collected = " + totalToll);

if (bikeCount > carCount && bikeCount > truckCount) {
    System.out.println("Most frequent vehicle = Bike");
} else if (carCount > bikeCount && carCount > truckCount) {
    System.out.println("Most frequent vehicle = Car");
} else if (truckCount > carCount && truckCount > bikeCount) {
    System.out.println("Most frequent vehicle = Truck");
} else {
    System.out.println("No single most frequent vehicle (there is a tie or no vehicles entered).");
}

input.close();
}
}
```

OUTPUT:

```
Enter vehicle number (or type 'done' to finish):
1
Enter vehicle type:
car
Enter vehicle number (or type 'done' to finish):
2
Enter vehicle type:
car
Enter vehicle number (or type 'done' to finish):
3
Enter vehicle type:
bike
Enter vehicle number (or type 'done' to finish):
4
Enter vehicle type:
truck
Enter vehicle number (or type 'done' to finish):
done

--- Summary ---
Total Toll Collected = 270
Most frequent vehicle = Car
```

PROBLEM-3:**Vending machine (VendingMachine.java, main in this file)**

- (a) Define an enum Coin with constants ONE, TWO, FIVE, TEN.
- (b) In main, set a snack price of 15 and a running total of 0; create a Scanner.
- (c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far.
- (d) Stop the loop once the total reaches 15 or more.
- (e) Print the change to return (total – 15).

Expected output: *TEN then FIVE prints “Paid. Change: 0”; TEN then TEN prints “Paid. Change: 5”.*

CODE:

```
import java.util.*;

class SnackDispenser {

    enum CoinType { ONE, TWO, FIVE, TEN }

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        int itemPrice = 15;
        int amountPaid = 0;

        while (amountPaid < itemPrice) {
            System.out.println("Enter the coin (ONE, TWO, FIVE, TEN):");
            String userInput = scanner.next().toUpperCase();

            CoinType insertedCoin = CoinType.valueOf(userInput);
            int coinValue = 0;

            switch (insertedCoin) {
                case ONE:
                    coinValue = 1;
                    break;
                case TWO:
                    coinValue = 2;
                    break;
                case FIVE:
                    coinValue = 3 + 2;
```

```
        break;
    case TEN:
        coinValue = 10;
        break;
    }

    amountPaid += coinValue;
    System.out.println("Current total inserted: " + amountPaid);
}

int changeDue = amountPaid - itemPrice;
System.out.println("Final Amount Paid = " + amountPaid);
System.out.println("Change Returned = " + changeDue);

scanner.close();
}
}
```

OUTPUT:

```
Enter the coin (ONE, TWO, FIVE, TEN):
ONE
Current total inserted: 1
Enter the coin (ONE, TWO, FIVE, TEN):
TWO
Current total inserted: 3
Enter the coin (ONE, TWO, FIVE, TEN):
TEN
Current total inserted: 13
Enter the coin (ONE, TWO, FIVE, TEN):
TWO
Current total inserted: 15
Final Amount Paid = 15
Change Returned = 0
```

=== Code Execution Successful ===

